

Guesty Short-Term Rental Property Management System

Product Requirements Document (PRD)

Document Version: 1.0

Date: December 30, 2025

Status: Reverse-Engineered Specification

Target Audience: Product Managers, Engineers, Designers, Investors

1. Executive Summary

Guesty is a cloud-based Property Management System (PMS) for short-term rentals that unifies operations across multiple booking channels (Airbnb, [Booking.com](#), Vrbo, and 60+ OTAs). The platform provides a single control plane for inventory management, dynamic pricing, guest communications, task automation, financial operations, and owner reporting.

Core Value Proposition:

- Eliminate channel fragmentation and reduce manual operational overhead by 50%+
- Enable data-driven revenue optimization through dynamic pricing and demand analytics
- Automate guest lifecycle and operations workflows (messaging, tasks, check-in/checkout)
- Provide compliant financial infrastructure (trust accounting, payouts, owner statements)
- Scale from solo hosts (1–3 listings) to enterprise portfolios (200+ properties)

Market Position: Enterprise-grade PMS with modular features, tiered pricing (Lite/Pro/Enterprise), and a broad integration marketplace.

2. Problem Statement & Opportunity

Current Pain Points

For Individual Hosts & Small Managers:

- Manual calendar management across multiple OTA channels leads to double bookings and blocked dates
- Fragmented guest communications across Airbnb messages, [Booking.com](#), email, and SMS require constant context switching
- Repetitive messaging (check-in, pre-arrival, post-checkout) consumes 5–10 hours/week per host
- Blind pricing decisions: no visibility into competitor rates, demand signals, or revenue optimization opportunities

- Operational chaos: cleaning schedules, maintenance, and guest coordination scattered across tools (WhatsApp, email, spreadsheets)
- Owner reporting: manual statements, unclear financial accounting, delayed payouts

For Enterprise & Brand Operators:

- No unified view of multi-brand, multi-region operations; separate dashboards per OTA
- Complex permission models for staff, cleaners, maintenance teams not supported
- Integration burden: custom API work to sync with accounting, lock systems, and cleaning services
- Capital constraints: difficulty in securing working capital or financing based on revenue visibility

Market Opportunity

- STR market growing 10–15% annually, with millions of active hosts globally
- Consolidation trend: hosts increasing portfolio size and seeking professional tools
- Increasing regulatory pressure: tax compliance, guest screening, liability management drive demand for integrated solutions
- Shift to direct bookings: hosts seeking to reduce OTA dependence and build guest loyalty

3. Target Users & Personas

3.1 Independent Host (Portfolio: 1–3 listings)

Name: Sarah (Guesty Lite user)

Goals:

- Minimize time spent on property management while maintaining high guest satisfaction
- Automate repetitive tasks (check-in, messaging)
- Understand revenue potential and optimize pricing

Painpoints:

- Limited technical skills; needs simplicity and quick setup
- Managing multiple calendars manually
- Struggling to respond to guest inquiries quickly

Usage Frequency: 2–5 hours/week

Device Preference: Mobile-first, browser-based for admin

3.2 Professional Manager (Portfolio: 4–199 listings)

Name: Marcus (Guesty Pro user)

Goals:

- Grow portfolio while maintaining operational control
- Data-driven pricing to maximize revenue
- Transparent owner reporting to maintain investor confidence

Painpoints:

- Team coordination challenges (cleaners, maintenance, guest comms)
- Complex financial tracking across multiple owners/properties
- Need for integration with accounting software (QuickBooks, Xero)

Usage Frequency: 4–6 hours/day

Device Preference: Desktop for analytics/reporting; mobile for on-the-go task management

3.3 Enterprise Portfolio Manager (Portfolio: 200+ listings)

Name: Jennifer (Guesty Enterprise user)

Goals:

- Manage multiple brands/regions from a single control plane
- Complex financial operations with tiered payouts, escrow, and compliance
- Leverage data for strategic decisions

Painpoints:

- Custom workflows and permission models needed
- Vendor management (cleaners, handymen, concierge services)
- Regulatory and tax compliance across jurisdictions
- Need for custom integrations and API-level control

Usage Frequency: 8+ hours/day (distributed team)

Device Preference: Desktop for strategic analytics; ecosystem integrations

3.4 Secondary Users

- **Property Owners/Investors:** View-only access to reservations, financials, and owner statements
- **Operations Team:** Task management, cleaning/maintenance assignment and execution
- **Guest Support Agent:** Unified messaging, guest CRM, and communications
- **Finance/Accounting:** Trust accounting, payout reconciliation, tax reporting

4. Product Vision & Strategic Goals

Vision Statement

To be the operating system for short-term rental operators globally, enabling transparent, data-driven, and automated property management at every scale.

Strategic Goals

1. Operational Efficiency

- Reduce manual operational hours by 50%+ through automation and consolidation
- Enable hosts to manage 5–10x more listings with same staffing

2. Revenue Optimization

- Drive 15–20% uplift in average daily rate (ADR) and revenue-per-available-room (RevPAR) through dynamic pricing
- Enable direct bookings to reduce OTA commission dependency

3. Scalability

- Support hosts from 1 listing to 1,000+
- Enterprise-grade infrastructure and API for custom integrations

4. Trust & Compliance

- Provide transparent financial accounting aligned with STR regulatory requirements
- Integrate guest verification and liability protection to reduce risk

5. Marketplace Ecosystem

- Foster third-party integrations for locks, cleaning services, pricing optimization, accounting
- Position Guesty as the hub for the STR tech stack

5. Core Product Modules

Module 1: Guests & Reservations

5.1.1 Unified Inbox

Overview:

Centralized messaging hub aggregating all guest communications across OTA channels (Airbnb, [Booking.com](#), Vrbo, email, SMS) into a single interface.

User Stories:

- As a host, I want to view all guest messages in one place so I can respond faster without switching between platforms
- As a support agent, I want to filter messages by reservation, guest, or property so I can prioritize urgent inquiries
- As a host, I want to see conversation history within a thread so I can maintain context across multiple exchanges

Functional Requirements:

- Aggregate messages from: Airbnb, [Booking.com](#), Vrbo, email, SMS, Guesty Guest App
- Display threaded conversations per reservation with timestamp and sender info
- Search and filter by: guest name, property, reservation ID, date range, keyword, unread status
- Auto-archiving of completed reservations; manual archive option
- Message history retention (cloud-based, searchable)
- Read/unread status tracking
- Notification management (in-app, email, mobile push)
- Rate limiting and spam detection

API Endpoints:

- GET /reservations/{id}/messages – Retrieve all messages for a reservation
- POST /messages – Send message across integrated channels
- GET /messages/search?q=keyword – Full-text search on messages
- PUT /messages/{id}/status – Mark as read/unread

- PATCH /reservations/{id}/archive – Archive conversation

Data Model:

Message {

id: UUID

reservation_id: UUID

guest_id: UUID

property_id: UUID

channel: enum [AIRBNB, BOOKING, VRBO, EMAIL, SMS, GUEST_APP]

direction: enum [INBOUND, OUTBOUND]

body: string

sender: {name, email, phone}

created_at: timestamp

read_at: timestamp

thread_id: UUID

ai_suggested_response?: string

}

Reservation {

id: UUID

guest_id: UUID

property_id: UUID

check_in: date

check_out: date

status: enum [PENDING, CONFIRMED, CHECKED_IN, COMPLETED, CANCELLED]

channel: enum [OTA, DIRECT, MANUAL]

occupants: integer

rate: decimal

currency: string

messages: Message[]

}

Automation & Triggers:

- Auto-archive after check-out + 7 days
- Flag messages >2 hours old without response
- Detect language and offer translation
- Trigger guest app notification when new message received

Permissions & Roles:

- Host: full read/write
- Support Agent: read/write (assigned reservations only)
- Owner: read-only
- Guest: read/write (guest app channel only)

Integrations:

- Airbnb API (webhooks for new messages)
- [Booking.com](https://www.booking.com) API
- Vrbo API
- Twilio (SMS)
- SendGrid/custom SMTP (email)

- Guesty Guest App

5.1.1.2 Multi-Calendar / Centralized Calendar

Overview:

Single unified view of all reservations across connected OTA channels and direct bookings, eliminating manual calendar management and preventing double bookings.

User Stories:

- As a host, I want to see all my properties' reservations in one calendar so I don't double-book
- As a manager, I want to block dates (owner stays, maintenance) so guests can't book those nights
- As a host, I want changes (dates, rates) synced automatically to all channels so I only update once
- As an operator, I want a month/week/day view to plan cleaning and maintenance

Functional Requirements:

- Multi-channel aggregation: all OTA + direct bookings visible in single calendar
- Display modes: month, week, day, grid (by property)
- Real-time conflict detection and prevention
- Drag-and-drop reservation modification (dates, property reassignment for multi-units)
- Date blocking: mark unavailable, owner stays, maintenance windows
- Color-coding by: channel, status (pending/confirmed/checked-in), property, guest type
- Notes/annotations per date
- Sync state indicator (pending, synced, sync error)
- Batch operations: block multiple dates, apply rate changes to date ranges

API Endpoints:

- GET /calendar?property_id=X&start_date=&end_date= – Retrieve calendar data
- POST /reservations – Create new reservation (manual entry)
- PATCH /reservations/{id} – Update reservation (dates, rates, notes)
- DELETE /reservations/{id} – Cancel reservation
- POST /calendar/block?property_id=X&start_date=&end_date= – Block dates
- POST /reservations/bulk?action= – Batch operations

Data Model:

```
CalendarEntry {  
  id: UUID  
  property_id: UUID  
  date: date  
  status: enum [AVAILABLE, BLOCKED, RESERVED, PENDING]  
  reservation_id?: UUID  
  block_reason?: enum [OWNER_STAY, MAINTENANCE, CLEANING_BUFFER, MANUAL]  
  notes: string  
  color_tag?: string  
}
```

```
ReservationSync {  
  id: UUID  
  reservation_id: UUID  
  channel: enum [AIRBNB, BOOKING, VRBO, DIRECT, MANUAL]  
  status: enum [IN_SYNC, PENDING_SYNC, SYNC_ERROR, STALE]  
  last_synced_at: timestamp  
  error_message?: string  
}
```

Automation & Triggers:

- Auto-sync rate/availability changes to all connected channels within 5 min
- Alert host if sync fails (with retry mechanism)
- Auto-block post-checkout cleaning buffer (configurable, e.g., 3 hours)
- Conflict detection: prevent overbooking if channel sync delay occurs

Permissions & Roles:

- Host: full read/write
- Manager: full read/write
- Support Agent: read-only
- Owner: read-only

Integrations:

- Airbnb API (real-time sync via webhooks)
- [Booking.com](#) API
- Vrbo API
- iCal feeds (import/export)
- Google Calendar (optional one-way export)

5.1.3 Guest App (White-Label Portal)

Overview:

Customizable, branded guest portal accessible via web/mobile link, providing check-in details, house manual, local guides, and upsell opportunities.

User Stories:

- As a guest, I want a personalized welcome portal with check-in instructions so I know what to do upon arrival
- As a host, I want to customize the app with my branding and house rules so guests feel the property's personality
- As a host, I want to upsell services (late checkout, cleaning, local experiences) to guests pre-arrival
- As a guest, I want access to WiFi passwords, emergency contacts, and local recommendations without asking the host

Functional Requirements:

- White-label customization: logo, colors, booking confirmation messaging
- Dynamic content injection based on: property, reservation dates, guest language
- Sections:

- Arrival details (check-in time, address, parking, codes/access info)
 - House manual (rules, appliances, WiFi, utilities, emergency procedures)
 - Contact info (host, property manager, emergency contact, local services)
 - Local guide (restaurants, attractions, maps)
 - Upsells (late checkout, cleaning add-on, experiences, activity packages)
 - Pre-arrival questionnaire (dietary restrictions, accessibility needs)
- Mobile-responsive design
- QR code to access (sent in pre-arrival email/SMS)
- Analytics: view tracking, section engagement, upsell conversion

API Endpoints:

- GET /guest-app/{reservation_id} – Fetch guest app content
- POST /guest-app/{reservation_id}/upsells – Log upsell impression/click
- POST /guest-app/{reservation_id}/questionnaire – Submit pre-arrival data

Data Model:

```

GuestApp {
  id: UUID
  property_id: UUID
  branding: {
    logo_url: string
    primary_color: hex
    secondary_color: hex
    custom_domain?: string
  }
  sections: {
    arrival: {enabled: bool, content: rich_text}
    house_manual: {enabled: bool, content: rich_text, attachments: File[]}
    contact_info: {enabled: bool, hosts: Contact[], emergency: Contact[]}
    local_guide: {enabled: bool, content: rich_text, poi: PointOfInterest[]}
    upsells: {enabled: bool, offers: Offer[]}
    questionnaire: {enabled: bool, questions: Question[]}
  }
  created_at: timestamp
  updated_at: timestamp
}

Upsell {
  id: UUID
  property_id: UUID
  type: enum [LATE_CHECKOUT, EARLY_CHECKIN, CLEANING, EXPERIENCE, ACTIVITY, OTHER]
  title: string
  description: string
  price: decimal
  image_url?: string
}

GuestAppAnalytics {
  reservation_id: UUID
  accessed_at: timestamp
}

```



```
sections_viewed: string[]
upsells_clicked: UUID[]
upsells_purchased: UUID[]
}
```

Automation & Triggers:

- Auto-send Guest App link 7 days before check-in
- Resend link 24 hours before check-in if not accessed
- Capture questionnaire responses and tag guest profile

Permissions & Roles:

- Host: full read/write (customize app for property)
- Guest: read-only during stay

Integrations:

- Email/SMS delivery of Guest App link
- Payment gateway (process upsell purchases)
- Analytics (Google Analytics, Mixpanel for tracking)

5.1.4 Guesty AI Suite™

Overview:

AI-powered messaging assistant that generates contextually appropriate guest responses, translates messages, and suggests next actions to reduce response time and improve guest satisfaction.

User Stories:

- As a host, I want AI to suggest responses to common guest inquiries (check-in time, Wi-Fi, directions) so I respond faster
- As a multilingual operator, I want messages automatically translated so I can respond to guests in their language
- As a busy host, I want AI to flag urgent messages (cancellation requests, maintenance issues) so I prioritize critical items
- As a support agent, I want AI to analyze guest sentiment to detect upset/at-risk guests so I escalate when needed

Functional Requirements:

- Message analysis: categorize inbound messages (question, issue, complaint, positive, checkout)
- Suggested response generation: context-aware, tone-matched to host communication style
- Multi-language support: detect language, translate to host's native language, provide response translation back
- Sentiment analysis: flag negative/urgent messages with priority indicator
- Learning from host responses: improve suggestions over time
- One-click response approval: host can send suggested response with optional edits
- Batch processing: process all pending messages nightly for suggestions
- Opt-in/opt-out per host

API Endpoints:

- POST /messages/{id}/ai-suggestions – Generate suggested responses
- POST /messages/{id}/ai-suggest/translate – Translate message + suggested response
- GET /messages/ai-priority-queue – Fetch flagged urgent messages
- POST /ai/feedback – Log host acceptance/rejection of suggestion (training signal)

Data Model:

Message {

...

ai_analysis: {

category: enum [QUESTION, ISSUE, COMPLAINT, POSITIVE, CHECKOUT, BOOKING_CHANGE, OTHER]

sentiment: enum [POSITIVE, NEUTRAL, NEGATIVE]

urgency: enum [LOW, MEDIUM, HIGH]

detected_language: string

key_entities: string[] # extracted topics, issues

}

ai_suggested_response?: string

ai_confidence_score?: float

host_feedback?: enum [ACCEPTED, EDITED, REJECTED]

}

AIPreference {

user_id: UUID

tone: enum [PROFESSIONAL, FRIENDLY, CASUAL]

language_preference: string

auto_suggest_enabled: bool

priority_alerts_enabled: bool

}

Automation & Triggers:

- Process new messages through AI pipeline within 30 seconds
- Queue suggestions for urgent messages (flag in UI)
- Auto-translate non-native language messages
- Weekly digest: summary of message trends and suggest improvements

Permissions & Roles:

- Host: view suggestions, opt-in/out
- Support Agent: view suggestions, send with approval
- Owner: no access

Integrations:

- OpenAI API (GPT-4/GPT-3.5-turbo for generation)
 - Google Translate API (translation)
 - Custom language models (fine-tuned on STR domain)
-

5.1.5 Guesty CRM

Overview:

Centralized guest profile database with booking history, preferences, communication record, and segmentation for targeted marketing and loyalty.

User Stories:

- As a host, I want to see a repeat guest's history so I can personalize their stay and offer preferred amenities
- As a manager, I want to segment guests (high-value, repeat, VIP) so I can target them with special offers
- As a host, I want to track guest preferences (late checkout, extra pillows, pet-friendly) so I deliver better experiences
- As a manager, I want to run email campaigns to past guests so I drive repeat bookings

Functional Requirements:

- Guest profile: amalgamated data from all channels (Airbnb, [Booking.com](#), direct)
- Unified identity: merge duplicate profiles across OTAs (fuzzy matching on email, phone, name)
- Booking history: all past/future reservations, cancellations, length of stay, rate paid
- Preferences & notes: dietary restrictions, accessibility needs, special requests, host notes
- Guest segmentation: auto-tags based on: spend, frequency, rating, channel, property type
- Communication history: all messages (not just Guesty) available per guest
- Lifecycle stage: first-time, repeat, VIP, at-risk (no booking >12 months)
- Integration with email/SMS for campaigns (via Mailchimp, HubSpot, etc.)
- Privacy compliance: GDPR consent tracking, data retention policies

API Endpoints:

- GET /guests/{id} – Fetch guest profile with history
- PATCH /guests/{id} – Update preferences, notes
- POST /guests/merge – Merge duplicate profiles
- GET /guests/segments – List segmented cohorts
- POST /guests/{id}/campaigns – Add guest to campaign

Data Model:

```
Guest {  
  id: UUID  
  email: string  
  phone?: string  
  name: string  
  photo_url?: string  
  created_at: timestamp  
  booking_count: integer  
  total_spent: decimal  
  avg_rating: float  
  languages: string[]  
  preferences: {
```

```
dietary_restrictions: string[]
accessibility_needs: string[]
pet_info?: string
special_requests?: string[]
}
host_notes: string
lifecycle_stage: enum [FIRST_TIME, REPEAT, VIP, AT_RISK, CHURNED]
segments: string[] # [high_value, weekend_warrior, etc.]
gdpr_consent: bool
communication_channel_preference?: enum [EMAIL, SMS, PHONE]
reservations: Reservation[]
messages: Message[]
}
```

```
GuestSegment {
id: UUID
name: string
description: string
criteria: {
min_booking_count?: integer
min_total_spent?: decimal
date_range?: {start, end}
min_rating?: float
properties?: UUID[]
channels?: enum[]
}
member_count: integer
created_at: timestamp
}
```

Automation & Triggers:

- Auto-tag new guest upon first booking
- Update lifecycle stage nightly (check for 12-month inactivity for at-risk)
- Send email when VIP guest books (flag for host)
- Merge duplicate profiles when new booking matches existing guest (fuzzy logic)

Permissions & Roles:

- Host: view profiles, edit preferences/notes for own guests
- Manager: full read/write, manage segments
- Support Agent: view profiles for assigned reservations
- Marketing: view segments, export for campaigns

Integrations:

- Mailchimp API (sync segments, send campaigns)
 - HubSpot API (contact sync)
 - Twilio (SMS campaigns)
 - Email providers (SendGrid, AWS SES)
 - Klaviyo (email marketing platform)
-

5.1.6 Direct Reservations

Overview:

Ability to create and manage reservations outside OTA channels (phone bookings, repeat guests, website direct bookings), with full integration into calendar, billing, and messaging.

User Stories:

- As a host, I want to accept a direct booking over the phone so I don't lose a sale to OTA friction
- As a manager, I want to offer a repeat guest a discount for booking direct so I reduce commission
- As a host, I want the direct booking to appear in my calendar and trigger automation just like OTA bookings
- As a manager, I want to collect payment for direct bookings so I minimize unpaid/disputed reservations

Functional Requirements:

- Create manual reservation with: guest name, email, phone, property, check-in/out, occupants, rate, currency
- Payment collection: card charge, manual payment link (Stripe, PayPal), bank transfer option
- Source tracking: phone, email, website, referral, other
- Integration with full Guesty stack: appears in calendar, inbox, task engine, analytics
- Refund management: partial/full refunds with reason tracking
- Auto-invoice generation (optional with tax number)
- Approval workflow: optional (for managers to approve large bookings)

API Endpoints:

- POST /reservations/direct – Create direct booking
- POST /reservations/{id}/payment – Collect payment
- POST /reservations/{id}/refund – Process refund
- GET /reservations/{id}/invoice – Generate invoice

Data Model:

```
DirectReservation {  
  id: UUID  
  guest: {name, email, phone}  
  property_id: UUID  
  check_in: date  
  check_out: date  
  occupants: integer  
  rate: decimal  
  currency: string  
  source: enum [PHONE, EMAIL, WEBSITE, REFERRAL, OTHER]  
  status: enum [PENDING, CONFIRMED, CHECKED_IN, COMPLETED, CANCELLED]  
  payment_status: enum [PENDING, PARTIAL, PAID, REFUNDED]  
  payment_method?: enum [CARD, BANK_TRANSFER, MANUAL]  
  notes: string  
  created_by: UUID  
  created_at: timestamp
```

```
updated_at: timestamp
}
```

```
Payment {
id: UUID
reservation_id: UUID
amount: decimal
currency: string
method: enum [CARD, ACH, MANUAL]
status: enum [PENDING, SUCCESS, FAILED, REFUNDED]
transaction_id?: string
created_at: timestamp
refunded_amount?: decimal
refund_reason?: string
}
```

Automation & Triggers:

- Create calendar entry and block dates immediately
- Send booking confirmation email/SMS to guest
- Trigger pre-arrival automation sequence
- Flag payment if status is "pending" after 3 days

Permissions & Roles:

- Host: create direct bookings for own properties
- Manager: full read/write across all properties
- Finance: view payment status, process refunds
- Owner: view as reservations (in their portal)

Integrations:

- Stripe (payment processing)
- PayPal (payment processing)
- Custom email/SMS (confirmation delivery)

5.1.7 Guest Communication Services™ (Managed Service)

Overview:

Optional white-glove service where Guesty's team manages guest communications 24/7 on behalf of hosts, reducing response burden and improving satisfaction.

User Stories:

- As a busy host, I want Guesty to handle initial guest inquiries so I can focus on property operations
- As a property manager, I want professional communications to reflect my brand so I delegate with confidence
- As a host, I want escalations to me for complex issues so I maintain control over property-specific decisions

Functional Requirements:

- Managed 24/7 support team responds to guest inquiries

- Response within: 1 hour (standard), 15 min (premium) depending on SLA
- Escalation criteria: property damage, cancellation requests, special accommodations
- Monthly reporting: response metrics, escalation reasons, guest satisfaction scores
- Agent access to: guest profile, booking details, house manual, host preferences/notes
- Integration point: messages routed through Unified Inbox, visibly marked as from support team
- Pricing: tiered by message volume, response SLA, or monthly fee

API Endpoints:

- POST /gcs/escalate – Flag for escalation to host
- GET /gcs/analytics?date_range= – Monthly performance report
- POST /gcs/preferences – Set escalation rules and communication guidelines

Data Model:

```
GCSEnrollment {
  user_id: UUID
  active: bool
  sla_tier: enum [STANDARD_1H, PREMIUM_15MIN]
  escalation_rules: {
    damage_report: bool
    cancellation_request: bool
    special_requests: bool
    complaint: bool
  }
  communication_guidelines: {
    tone: string
    languages: string[]
    brand_guidelines: string
  }
  assigned_agents: User[]
  created_at: timestamp
}
```

```
GCSMessage {
  message_id: UUID
  handled_by: enum [HOST, GCS_AGENT]
  agent_id?: UUID
  escalated_to_host: bool
  escalation_reason?: string
  response_time_seconds: integer
  created_at: timestamp
}
```

Automation & Triggers:

- Route new inquiry to GCS team if service is active
- Set timer for SLA response deadline
- Auto-escalate if response SLA breached
- Generate daily digest of escalations for host

Permissions & Roles:

- Host: view all communications (marked if from GCS)
- GCS Agent: access to assigned reservations only
- Manager: view GCS metrics across portfolio

Integrations:

- Internal Guesty agent platform (CRM, knowledge base, escalation system)
-

5.1.8 Manual Reservations**Overview:**

Back-office tool to create and manage reservations in bulk, useful for migration from other PMSs, group bookings, or special arrangements.

User Stories:

- As a manager, I want to bulk-import reservations from my old PMS so I don't re-enter data manually
- As a host, I want to create a multi-unit group booking so I can assign different units to one guest party

Functional Requirements:

- Manual creation interface: date range, multi-unit selection, auto-calculate occupancy/rate
- Bulk import: CSV template with validation, duplicate detection
- Special handling: group bookings (split across units), owner stays
- Auto-sync to calendar and sync to channels (if applicable)

API Endpoints:

- POST /reservations/bulk/import – Bulk upload via CSV
 - POST /reservations/group – Create group booking
-

5.1.9 Damage Protection (Add-on)**Overview:**

Insurance product protecting hosts from guest-caused damage, integrated into booking workflow with claims management.

User Stories:

- As a host, I want to add damage protection to each booking so I'm covered if a guest damages my property
- As a guest, I want transparent pricing for damage protection so I know what I'm insuring
- As a host, I want to file a damage claim quickly with photos so I get reimbursed promptly

Functional Requirements:

- Offer damage protection at booking time (add-on checkbox)

- Pricing: fixed per-night or percentage of booking value
- Integration with guest app: upsell damage protection at booking confirmation
- Claims portal: upload photos, describe damage, attach repair estimates
- Claims workflow: host submits → Guesty/insurer reviews → approval/denial
- Payout to host: either direct reimburse or payment to repair vendor

API Endpoints:

- POST /bookings/{id}/add-on/damage-protection – Add protection to booking
- POST /claims – File damage claim
- GET /claims/{id}/status – Check claim status

Data Model:

```
DamageProtection {
reservation_id: UUID
protection_type: enum [STANDARD, PREMIUM]
price: decimal
currency: string
coverage_limit: decimal
status: enum [ACTIVE, INACTIVE, CLAIMED, EXPIRED]
}
```

```
DamageClaim {
id: UUID
reservation_id: UUID
filed_date: timestamp
description: string
photos: File[]
repair_estimates: File[]
amount_claimed: decimal
status: enum [PENDING, UNDER_REVIEW, APPROVED, REJECTED, PAID]
decision_date?: timestamp
payout_amount?: decimal
payout_date?: timestamp
}
```

5.1.10 Guesty Verify (Guest Verification)

Overview:

Pre-check-in identity verification and risk-scoring of guests, reducing fraud and no-shows.

User Stories:

- As a host, I want to verify guest identity before check-in so I reduce fraud and damage risk
- As a host, I want to see a risk score for bookings so I can decide whether to accept the reservation
- As a guest, I want a quick, one-click ID verification so I don't delay the booking process

Functional Requirements:

- Pre-booking or pre-check-in ID verification (selectable by host)

- Identity verification: government ID scan + face verification (Liveness check)
- Risk scoring: 1–100 score based on: verification result, guest history, dispute record, review ratings
- Fraud flag: flag risky bookings for manual review
- Integration: offer during booking flow (guest app) or before check-in
- Compliance: GDPR-compliant data storage and retention

API Endpoints:

- POST /guests/{id}/verify – Initiate verification
- GET /guests/{id}/risk-score – Retrieve risk assessment
- POST /reservations/{id}/verify-gate – Gate check-in until verified

Data Model:

```
GuestVerification {
  guest_id: UUID
  verification_status: enum [PENDING, IN_PROGRESS, VERIFIED, FAILED, EXPIRED]
  id_type: enum [PASSPORT, DRIVERS_LICENSE, NATIONAL_ID]
  id_country: string
  verified_at?: timestamp
  liveness_check_passed?: bool
  risk_score: integer # 0–100
  risk_level: enum [LOW, MEDIUM, HIGH]
  risk_factors: string[]
  expires_at?: timestamp
}
```

Automation & Triggers:

- Auto-initiate verification for high-risk properties (high turnover, urban locations)
- Alert host if risk score > 70
- Block check-in if verification fails and host has required verification

Permissions & Roles:

- Host: configure verification requirements, view risk scores
- Guesty: manage verification provider integrations

Integrations:

- Third-party identity verification provider (e.g., IDology, Onfido, Jumio)

Module 2: Distribution & Operations

5.2.1 Channel Manager

Overview:

Core inventory and rate distribution across 60+ OTA channels (Airbnb, [Booking.com](#), Vrbo, Expedia, etc.), with real-time sync to prevent double bookings and enable dynamic pricing.

User Stories:

- As a host, I want to manage my listing on Airbnb, [Booking.com](#), and Vrbo from one place so I don't manually sync rates

- As a manager, I want to set min/max prices and length-of-stay rules that automatically apply across all channels
- As a revenue manager, I want to adjust rates in real-time based on demand without logging into each OTA

Functional Requirements:

- List aggregation: connect multiple OTA accounts (hosts often have separate Airbnb accounts, [Booking.com](#) IDs, etc.)
- Content sync: title, description, photos, amenities sync to channels
- Availability & rate distribution:
 - Master availability calendar synced to all channels
 - Rate card management: set rates per night, min/max prices, length-of-stay minimums/maximums
 - Blackout dates and seasonal pricing rules
 - Real-time sync: rates/availability update across channels within 5 minutes
- Sync status monitoring: track which updates succeeded/failed, retry logic
- OTA-specific handling: Airbnb's preferred partner program optimization, [Booking.com](#)'s commission changes, etc.
- Preferred partner tools: enable commissions, performance metrics, and special features per OTA
- Bulk rate updates: apply rate % change across date ranges or properties

API Endpoints:

- POST /channels/connect – Authorize and connect new OTA account
- GET /channels/connected – List connected channels per property
- PATCH /listings/{id}/rates?channel=AIRBNB – Update rate for channel
- POST /listings/{id}/sync – Force immediate sync to all channels
- GET /sync-status/{id} – Check sync status and errors

Data Model:

ListingSync {

id: UUID

property_id: UUID

channel: enum [AIRBNB, BOOKING, VRBO, EXPEDIA, AIRBNB_PLUS, VRBO_PLUS, ...]

channel_listing_id: string # Airbnb ID, [Booking.com](#) ID, etc.

sync_status: enum [IN_SYNC, PENDING, ERROR, PAUSED]

last_synced: timestamp

error_message?: string

content: {

title: string

description: string

photos: string[]

amenities: string[]

}

}

RateCard {

id: UUID

property_id: UUID

channel?: enum # NULL = master rate, otherwise channel-specific override

```
date_from: date
date_to: date
nightly_rate: decimal
min_price?: decimal
max_price?: decimal
los_min?: integer # length of stay minimum
los_max?: integer # length of stay maximum
weekend_modifier?: float # e.g., 1.2 = 20% increase for Fri-Sat
currency: string
}
```

```
ChannelConnection {
id: UUID
user_id: UUID
channel: enum
channel_account_id: string
oauth_token: encrypted_string
connected_at: timestamp
status: enum [ACTIVE, DISCONNECTED, ERROR, REFRESH_NEEDED]
}
```

Automation & Triggers:

- Hourly sync of rates/availability to channels
- Alert if sync fails 3+ consecutive times
- Auto-reconnect if OAuth token refresh fails
- Apply dynamic pricing rules automatically every night

Permissions & Roles:

- Host: view connected channels, manage rates for own properties
- Manager: full read/write across all connected channels
- Revenue Manager: modify rates and rules
- Support: read-only view

Integrations:

- Airbnb API v2 (OAuth, listing search, rates, reservations)
- [Booking.com](https://booking.com) API (Booking API v2)
- Vrbo/HomeAway API
- Expedia API
- Plus 50+ niche OTA APIs (Airbnb Plus, 9flats, Stayz, etc.)

5.2.2 Guesty Websites (Direct Booking Site)

Overview:

Branded, SEO-optimized direct booking website builder enabling hosts to accept bookings directly, reducing OTA commission and building guest loyalty.

User Stories:

- As a host, I want a professional website for my properties so guests can book directly without OTA fees

- As a manager, I want my website to rank on Google so I capture organic traffic
- As a host, I want to set different rates for website bookings so I incentivize direct bookings

Functional Requirements:

- Website builder: drag-and-drop interface (or template-based) for non-technical hosts
- Design templates: multiple property type templates (apartment, villa, cottage, etc.)
- Customization: colors, fonts, header image, custom domain (CNAME)
- Content syncing: auto-pull property data (photos, description, amenities) from Guesty
- Multi-listing site: showcase portfolio of properties on one domain
- SEO optimization: meta tags, Open Graph, sitemap.xml, fast load times
- Booking engine:
 - Date picker with live availability from Guesty calendar
 - Guest info collection form (name, email, phone, occupants, special requests)
 - Payment gateway: Stripe, PayPal, or custom PSP integration
 - Confirmation email with booking details and guest app link
- Analytics: track visitors, conversion rate, bookings sourced from website
- A/B testing: test different headline/offer variants
- Email capture: optional email signup for newsletter

API Endpoints:

- POST /websites – Create new website
- PATCH /websites/{id} – Update website settings/design
- GET /websites/{id}/bookings – Retrieve website-sourced bookings
- GET /websites/{id}/analytics – Website traffic and conversion metrics

Data Model:

```

Website {
  id: UUID
  user_id: UUID
  domain_custom?: string # e.g., myproperties.com
  domain_guesty?: string # e.g., username.stays.guesty.com
  theme: enum [MODERN, MINIMALIST, LUXURY, BOUTIQUE, ...]
  branding: {
    header_image: URL
    logo: URL
    primary_color: hex
    secondary_color: hex
  }
  properties: UUID[]
  seo: {
    meta_title: string
    meta_description: string
    canonical_url: string
  }
  cta_text: string # Call-to-action button text
  email_capture_enabled: bool
  payment_methods: enum[] # [STRIPE, PAYPAL, BANK_TRANSFER]
  created_at: timestamp

```

```
published: bool
}
```

```
WebsiteAnalytics {
  website_id: UUID
  date: date
  visitors: integer
  sessions: integer
  bookings: integer
  conversion_rate: float
  revenue: decimal
}
```

Automation & Triggers:

- Auto-update property availability in website when calendar changes
- Send booking confirmation email automatically
- Sync website booking to Guesty reservation system

Permissions & Roles:

- Host: manage own website
- Manager: manage websites for all properties
- Designer: customize templates (internal Guesty role)

Integrations:

- Stripe (payment processing)
- PayPal (payment processing)
- SendGrid (email confirmations)
- Google Analytics (traffic tracking)
- Cloudflare (DNS, CDN for custom domain)

5.2.3 Task Management

Overview:

Automated task generation and assignment system for cleaning, maintenance, guest services, and operations based on reservation events and rules.

User Stories:

- As a manager, I want cleaning tasks auto-created after checkout so my cleaner knows what to do
- As a cleaner, I want tasks assigned to me with photos and checklists so I know exactly what to clean
- As a manager, I want to track task progress so I ensure quality and on-time completion
- As a host, I want to assign maintenance tasks when guests report issues so they're addressed promptly

Functional Requirements:

- Auto-task generation:
 - Post-checkout cleaning (customizable by property, cleaning time budget)

- Pre-arrival inspection/preparation
 - Maintenance on-demand (host/guest-triggered)
 - Linen change, restocking consumables
 - Deep clean (monthly, quarterly rules)
- Task templates: create custom templates for recurring workflows (e.g., "Weekend turnover")
- Task details: title, description, due date/time, priority, checklist items, photos, attachments
- Assignment: to staff, vendor (cleaner, handyman, concierge), or automated via Zapier
- Mobile app: receive task assignment, photo checklist, mark complete
- Status tracking: not started, in progress, completed, quality check required, failed
- Photo requirement: require photos before task marked complete
- Time tracking: optional time-on-task logging for labor metrics
- Dependencies: create task chains (e.g., cleaning must complete before guest check-in)
- Priority management: urgency flags, SLA timers (e.g., flag if maintenance task unpicked after 2 hours)

API Endpoints:

- POST /tasks – Create manual task
- POST /tasks/templates/{id}/apply – Apply task template to reservation
- PATCH /tasks/{id} – Update task status, assign to staff
- POST /tasks/{id}/complete – Mark task complete with photos
- GET /tasks/staff/{staff_id} – Retrieve tasks assigned to person
- GET /properties/{id}/task-analytics – Tasks summary per property

Data Model:

```
Task {
id: UUID
property_id: UUID
reservation_id?: UUID
trigger: enum [CHECKOUT, CHECKIN, MANUAL, MAINTENANCE_REQUEST, TEMPLATE]
type: enum [CLEANING, MAINTENANCE, INSPECTION, RESTOCKING, LINEN, DEEP_CLEAN, OTHER]
title: string
description: string
due_date: timestamp
priority: enum [LOW, NORMAL, HIGH, URGENT]
status: enum [UNASSIGNED, ASSIGNED, IN_PROGRESS, COMPLETED, QUALITY_CHECK, FAILED]
assigned_to: UUID[] # staff/vendor IDs
checklist_items: {
item: string
completed: bool
photo_required: bool
}[]
photos: {
url: URL
uploaded_at: timestamp
uploaded_by: UUID
}
```

```

>[]
sla_deadline?: timestamp
time_on_task_minutes?: integer
notes: string
created_at: timestamp
completed_at?: timestamp
}

TaskTemplate {
id: UUID
property_id: UUID
name: string
description: string
type: enum
default_priority: enum
checklist_items: string[]
estimated_duration_minutes: integer
recurrence?: enum [ONE_TIME, WEEKLY, MONTHLY, QUARTERLY]
}

```

```

Staff {
id: UUID
user_id: UUID
name: string
email: string
phone: string
role: enum [CLEANER, HANDYMAN, CONCIERGE, INSPECTOR, MANAGER]
properties: UUID[] # Which properties this person works for
phone_verified: bool
availability_calendar: {start_time, end_time, available_days}
task_count_active: integer
}

```

Automation & Triggers:

- Auto-create cleaning task 2 hours before checkout (configurable cleanup time)
- Auto-create pre-arrival task 24 hours before check-in
- Alert manager if task not started 1 hour before due date
- Auto-escalate to manager if staff unresponsive (task unstarted after 2 hours)
- Create follow-up inspection task after cleaning completion (optional)

Permissions & Roles:

- Manager: create, assign, view all tasks
- Staff/Vendor: view assigned tasks, update status, upload photos
- Host: create ad-hoc tasks, view task status
- Owner: view-only (summary dashboard)

Integrations:

- Mobile app (task push notifications, photo capture)
- Zapier (route tasks to external platforms like Slack, Asana)
- SMS/email notifications (task assignment and deadline reminders)

- Google Calendar export (optional, for staff to see tasks in calendar view)
-

5.2.4 Automation Tools

Overview:

Rules engine enabling hosts to define "if-this-then-that" workflows that trigger actions based on reservation events, reducing manual work and improving consistency.

User Stories:

- As a host, I want to auto-send pre-arrival check-in instructions so I don't repeat myself
- As a manager, I want to apply price overrides during events (e.g., +30% during holidays) without manual updates
- As a host, I want to auto-assign cleaning tasks after checkout so there's no delay
- As a revenue manager, I want to auto-adjust rates based on occupancy so I maximize revenue

Functional Requirements:

- Visual rules builder: if [trigger] then [action], with AND/OR logic
- Triggers (events):
 - Reservation created/confirmed/cancelled
 - Check-in time (24h before, 12h before, at arrival)
 - Check-out time (24h before, at departure)
 - Payment received
 - Message received from guest
 - Review posted
 - Occupancy threshold (e.g., >80% booked)
 - Date-based (specific dates, holidays, seasons)
- Actions:
 - Send message (SMS, email, guest app notification) with template variables
 - Adjust pricing (add %, set fixed price, apply seasonal modifier)
 - Assign task (cleaning, inspection, etc.)
 - Apply tag (to reservation, guest, property)
 - Trigger external action via Zapier (Slack notification, Asana task, Google Sheet update)
 - Add to guest/reservation notes
 - Request review (prompt guest to leave review post-checkout)
- Template variables: {{guest_name}}, {{check_in_date}}, {{property_name}}, {{price}}, etc.
- Scheduling: run immediately, delayed (e.g., 1 hour before check-in), or batch nightly
- Testing: preview rules and test with sample data before activation
- Rule library: pre-built templates (e.g., "Welcome message", "Late checkout upsell")

API Endpoints:

- POST /automation-rules – Create new automation rule
- PATCH /automation-rules/{id} – Update rule
- POST /automation-rules/{id}/test – Test rule with sample data
- POST /automation-rules/{id}/activate – Enable rule
- GET /automation-rules – List all rules for user

Data Model:

```
AutomationRule {
  id: UUID
  user_id: UUID
  property_ids: UUID[] # Apply to specific properties or all
  name: string
  description: string
  active: bool
  trigger: {
    event: enum [RESERVATION_CREATED, RESERVATION_CONFIRMED, CHECKIN_24H,
    CHECKOUT_24H, PAYMENT_RECEIVED, MESSAGE_RECEIVED, REVIEW_POSTED,
    OCCUPANCY_THRESHOLD, DATE_BASED, ...]
    event_params: {
      occupancy_threshold?: float
      specific_dates?: date[]
      time_offset?: {value: integer; unit: enum [HOURS, DAYS]}
    }
  }
  conditions: {
    logic: enum [AND, OR]
    filters: {
      field: enum [GUEST_TYPE, CHANNEL, DURATION, RATE, OCCUPANCY, ...]
      operator: enum [EQUALS, CONTAINS, GREATER_THAN, LESS_THAN, ...]
      value: string | number
    }[]
  }?
  actions: {
    action_type: enum [SEND_MESSAGE, ADJUST_PRICE, CREATE_TASK, ADD_TAG,
    ZAPIER_TRIGGER, REQUEST_REVIEW]
    params: {
      message_template?: string
      message_type?: enum [SMS, EMAIL, PUSH, GUEST_APP]
      price_modifier?: {type: enum [PERCENTAGE, FIXED], value: number}
      task_type?: string
      tags?: string[]
      zapier_webhook?: URL
      review_delay_days?: integer
    }
  }[]
  created_at: timestamp
  updated_at: timestamp
}
```

```
AutomationExecution {
  id: UUID
  rule_id: UUID
  reservation_id: UUID
  executed_at: timestamp
  status: enum [SUCCESS, FAILED, PENDING]
  error_message?: string
  actions_executed: {
```

```
action: string
result: string
}[]
}
```

Automation & Triggers:

- Rules engine runs every 5 minutes (check for time-based triggers)
- Reservation events trigger rules immediately (check-in, payment, etc.)
- Store execution log for debugging and auditing

Permissions & Roles:

- Host: create and manage rules for own properties
- Manager: manage rules across portfolio
- Support: read-only

Integrations:

- Zapier (route actions to hundreds of third-party tools)
- Twilio (SMS delivery)
- SendGrid (email delivery)
- Slack (notifications)
- Google Sheets (append data)

5.2.5 Multi-Unit Management

Overview:

Specialized handling for buildings with multiple units (apartment complexes, hostels, short-term rental buildings) with shared facilities and combined reporting.

User Stories:

- As a manager of a 10-unit building, I want to manage all units from one dashboard so I have a unified view
- As a manager, I want to set shared amenities (gym, pool, parking) that apply to multiple units
- As a manager, I want to combine revenue reporting by building so I see total property performance
- As a guest, I want to see the same building as offering multiple room types so I choose the right one

Functional Requirements:

- Property hierarchy: building > units > bedrooms/spaces
- Shared amenities: define once, apply to multiple units (pool, gym, parking, security, WiFi quality/speed)
- Unit-specific amenities: distinguish unit-level features (kitchen, living space, balcony, AC, etc.)
- Combined calendar: view all units for a building in a single grid view
- Rate card management: set per-unit rates and bulk rate changes across units
- Revenue aggregation: combine revenue reporting for building vs. unit view
- Guest experience: show unit options clearly in Guesty Website

- Cleaning/maintenance: assign tasks to shared facilities and unit-specific spaces
- Building-level analytics: occupancy by unit, revenue by unit, seasonal trends for entire building

API Endpoints:

- POST /properties/building – Create building hierarchy
- PATCH /properties/{id}/shared-amenities – Update shared amenities
- GET /properties/{id}/units – List all units in building
- GET /building/{id}/analytics – Combined analytics for building

Data Model:

Building {

id: UUID

name: string

address: string

user_id: UUID

units: Property[]

shared_amenities: {

name: string

icon: string

description: string

}[]

shared_photos: URL[]

}

Property {

id: UUID

building_id?: UUID # If part of building

name: string

bedrooms: integer

bathrooms: integer

max_occupants: integer

unit_specific_amenities: string[]

photos: URL[]

enabled: bool

}

Permissions & Roles:

- Manager: manage entire building and all units
- Unit-level staff: manage assigned unit only

5.2.6 Analytics & Reporting Tools

Overview:

Comprehensive dashboards and exportable reports providing insights into occupancy, revenue, channel performance, and operations.

User Stories:

- As a host, I want to see my occupancy rate and revenue so I understand property performance

- As a manager, I want to track RevPAR and ADR trends so I adjust pricing strategy
- As an owner, I want to see detailed financial statements so I verify manager's performance
- As a revenue manager, I want to compare performance across properties so I identify optimization opportunities

Functional Requirements:

- Dashboards:
 - Executive summary: total revenue YTD, occupancy %, avg ADR, upcoming occupancy (30 days)
 - Property performance: revenue by property, occupancy by property, seasonal patterns
 - Channel performance: revenue by channel, booking count by channel, avg length of stay per channel
 - Financial summary: gross revenue, fees/commissions paid, net revenue, payouts processed
 - Operations: task completion rate, avg response time (messages), guest satisfaction scores
- KPI cards: customizable widgets for at-a-glance metrics
- Filters: date range, property, channel, guest type, source
- Reports (exportable as PDF/CSV):
 - Revenue report: daily/weekly/monthly revenue breakdown by property/channel
 - Occupancy report: occupancy by date, duration distribution, gap analysis
 - Guest report: guest count, booking sources, top channels, repeat guest %
 - Financial statement: revenue, expenses, owner payouts
 - Tax report: revenue by jurisdiction, guest taxes collected, expense categories
 - Operations report: task completion, response times, incident log
- Benchmarking: compare performance vs. previous period, vs. market (if anonymized data available)
- Custom reports: allow hosts to define custom report templates
- Scheduled reports: auto-email reports weekly/monthly
- Data export: raw data export (CSV) for further analysis

API Endpoints:

- GET /analytics/dashboard – Fetch dashboard data
- GET /analytics/revenue?property_id=X&date_range= – Revenue analytics
- GET /analytics/occupancy?date_range= – Occupancy metrics
- POST /reports/generate – Generate custom report
- GET /reports/schedule – List scheduled reports

Data Model:

AnalyticsSnapshot {

id: UUID

user_id: UUID

date: date

property_id?: UUID

channel?: enum

metrics: {

revenue_gross: decimal

```

revenue_net: decimal
occupancy_count: integer
occupancy_rate: float
available_nights: integer
booked_nights: integer
adp: float # Average Daily Rate
revpar: float # Revenue Per Available Room
booking_count: integer
cancellation_count: integer
avg_los: float # Length of Stay
repeat_guest_count: integer
}
}

ReportSchedule {
id: UUID
user_id: UUID
report_type: enum [REVENUE, OCCUPANCY, FINANCIAL, TAX, OPERATIONS, CUSTOM]
frequency: enum [WEEKLY, MONTHLY, QUARTERLY]
email_recipients: string[]
custom_filters?: {property_id, channel, date_range}
created_at: timestamp
}

```

Automation & Triggers:

- Nightly snapshot of metrics for historical tracking
- Auto-generate scheduled reports and send emails

Permissions & Roles:

- Host: view own property dashboards and reports
- Manager: view portfolio dashboards
- Owner: view financial reports (assigned properties only)
- Finance: view all financial reports

Integrations:

- Google Sheets (export data for further analysis)
- Data warehouse (Snowflake, BigQuery) for advanced analytics tools
- BI tools (Tableau, Looker) for custom dashboards

5.2.7 Guesty LocksManager™

Overview:

Integration hub for smart locks and keyless entry systems, with automatic access code generation and delivery to guests.

User Stories:

- As a host, I want guests to receive unique access codes automatically so I don't need to hide keys

- As a property manager, I want to revoke guest access automatically at checkout so I don't need manual key returns
- As a guest, I want to receive a simple code via SMS or app so I can access the property easily

Functional Requirements:

- Supported lock systems: integrate with major smart lock providers (August, Level Lock, Nuki, etc.)
- Access code management:
 - Auto-generate unique code per reservation
 - Set code duration: active from check-in to checkout + buffer (e.g., +1 hour)
 - Auto-delete code after expiration
 - Manual code revocation if needed
- Code delivery:
 - Send code via SMS (Twilio)
 - Send code via guest app
 - Send code via email with access instructions
 - Resend if guest lost code
- Integration with Guesty systems:
 - Auto-code generation triggered by reservation confirmed
 - Early check-in: extend code availability if early arrival requested
 - Late checkout: extend code if late checkout approved
 - Cancellation: auto-revoke codes
- Fallback access: maintain override access for emergencies (manager/owner)
- Activity log: track who accessed when (audit trail)

API Endpoints:

- POST /locks/authorize – Connect lock provider account
- POST /reservations/{id}/locks/generate-code – Generate access code
- DELETE /locks/{id}/code/{code} – Revoke access code
- GET /locks/{id}/activity-log – View access history

Data Model:

```
SmartLock {
  id: UUID
  property_id: UUID
  lock_type: enum [AUGUST, LEVEL_LOCK, NUKI, YALE_CONNECT, FRIDAY, ...]
  lock_device_id: string
  connection_status: enum [CONNECTED, DISCONNECTED, ERROR]
  last_sync: timestamp
}
```

```
AccessCode {
  id: UUID
  lock_id: UUID
  reservation_id: UUID
  code: string
  active_from: timestamp
  active_until: timestamp
  delivered: bool
}
```

```
delivery_method: enum [SMS, EMAIL, GUEST_APP]
status: enum [ACTIVE, EXPIRED, REVOKED]
created_at: timestamp
}
```

```
AccessLog {
id: UUID
lock_id: UUID
access_time: timestamp
accessed_by: enum [GUEST_CODE, MANAGER_OVERRIDE, SYSTEM]
code_id?: UUID
success: bool
}
```

Automation & Triggers:

- Auto-generate code when reservation confirmed
- Auto-extend code for early check-in (if requested/approved)
- Auto-revoke code at checkout + buffer period
- Alert manager if access log shows suspicious activity

Permissions & Roles:

- Host: configure lock, view access logs
- Manager: configure locks across properties, view all access logs
- Guest: receive and use code (no portal access)

Integrations:

- August API
- Level Lock API
- Nuki API
- Yale Connect API
- Friday API
- Twilio (SMS delivery)

5.2.8 Mobile App

Overview:

Native iOS/Android app providing essential property management functionality on-the-go, mirroring key desktop features.

User Stories:

- As a host, I want to check reservations and messages on my phone so I can respond quickly even when away
- As a cleaner, I want to receive task assignments and upload photos via app so I don't need laptop
- As a manager, I want to check property stats while traveling so I'm always informed

Functional Requirements:

- Core features (iOS/Android):
 - Calendar view: upcoming reservations, dates, guest info

- Unified Inbox: read/respond to messages, with notification badges
- Tasks: assigned tasks, task completion with photo upload
- Notifications: real-time alerts for bookings, messages, task reminders
- Reporting: quick dashboard with key metrics (revenue, occupancy)
- Settings: profile, notification preferences, logout
- Offline capability: basic data cached, sync when online
- Biometric login: fingerprint/face ID for security
- Push notifications: customizable alerts for bookings, messages, tasks
- Photo upload: use device camera or library for task photos

Technical Details:

- Platform: iOS (Swift/SwiftUI) and Android (Kotlin)
- Back-end: REST API + WebSocket for real-time updates
- Local storage: encrypted SQLite for offline data
- Push: Firebase Cloud Messaging (FCM) for Android, APNs for iOS

API Endpoints:

- Core endpoints shared with web app (REST)
- WebSocket endpoint for real-time updates: /ws/user/{id}

5.2.9 Enterprise Management Hub

Overview:

Centralized control plane for large portfolios with multi-brand, multi-region operations, advanced permissions, and consolidated reporting.

User Stories:

- As an enterprise manager, I want to manage multiple brands separately so each brand has its own identity and P&L
- As a regional manager, I want to see only my region's properties so I don't get distracted by other regions
- As a corporate user, I want consolidated revenue across all brands so I report to board
- As a finance team member, I want to access consolidated payouts and accounting so I process payments centrally

Functional Requirements:

- Organization structure:
 - Parent organization > brands/divisions > properties
 - Sub-accounts: separate login for each brand manager; filtered by brand
 - Permissions hierarchy: corporate > regional > brand > property level
- Consolidated reporting:
 - View revenue, occupancy, performance across all properties
 - Filter by brand, region, property type
 - Drill-down capability: click a brand to see properties within
- User roles & permissions:
 - Enterprise Admin: full access across all brands
 - Brand Manager: full access to assigned brand only
 - Regional Manager: access to properties in assigned region

- Finance: read-only access to financial reporting
 - Support Agent: limited access (inbox, tasks)
- Consolidated payout management:
 - Collect payouts from all brands
 - Batch process payments
 - Consolidated accounting records
- Integration management: manage API keys, integrations at enterprise level

API Endpoints:

- POST /organization/brands – Create new brand
- POST /organization/permissions – Assign role/permissions
- GET /organization/consolidated-analytics – Enterprise-wide reporting
- GET /organization/payouts – Consolidated payout records

Data Model:

Organization {

id: UUID

name: string

owner_user_id: UUID

subscription_plan: enum [LITE, PRO, ENTERPRISE]

brands: Brand[]

}

Brand {

id: UUID

organization_id: UUID

name: string

logo_url?: string

properties: Property[]

}

Permissions {

id: UUID

user_id: UUID

role: enum [ENTERPRISE_ADMIN, BRAND_MANAGER, REGIONAL_MANAGER, FINANCE, SUPPORT]

scope: {

organization_id?: UUID

brand_id?: UUID

property_ids?: UUID[]

region?: string

}

}

5.2.10 Shield Suite & Liability Coverage

Overview:

Optional risk-mitigation bundle including guest screening (Verify), damage protection, liability insurance, and proactive alerts.

User Stories:

- As a host, I want comprehensive coverage against guest damage and liability so I'm protected
- As a host, I want to know if a guest is risky before accepting so I can decline high-risk bookings

Functional Requirements:

- Bundle components:
 - Guesty Verify: pre-arrival guest screening
 - Damage Protection: guest damage coverage
 - Liability Coverage: host liability for guest injuries
 - Alerts: flagging of at-risk bookings and incidents
 - Claims management: unified dashboard for filing, tracking, and settling claims
 - Integration with insurance providers for underwriting and payouts
-

Module 3: Business & Financials

5.3.1 Revenue Management & Guesty PriceOptimizer™

Overview:

Dynamic pricing engine that optimizes nightly rates based on demand signals, competitor pricing, occupancy, events, and length of stay, with automated application across all channels.

User Stories:

- As a host, I want prices to adjust automatically based on demand so I maximize revenue without manual updates
- As a revenue manager, I want to see competitor rates so I can price competitively
- As a host, I want to set price floors and caps so prices don't get too low or high
- As a manager, I want to apply event-based pricing (festival, sports event, holidays) so I capture demand spikes

Functional Requirements:

- Demand signals:
 - Occupancy trend: detect high-demand periods by analyzing bookings
 - Seasonal patterns: learn from historical data (same dates last year)
 - Day-of-week: differentiate weekday vs. weekend pricing
 - Lead time: adjust based on how far in advance bookings are coming
- Pricing rules:
 - Base rate: starting point
 - Min/max price: hard boundaries
 - Length of stay (LOS) discounts: lower rates for 7+ nights, 30+ nights
 - Early-bird discount: lower rates for bookings far in advance
 - Last-minute boost: higher rates for bookings < 7 days out
 - Event-based modifiers: +30% during holidays, festivals, sporting events
 - Occupancy-based: raise rates when nearby properties highly booked, lower when low occupancy
- Competitor pricing:
 - Monitor rates of similar properties in area (Airbnb Comparable)
 - Auto-adjust to stay within 10% of market (configurable)

- Automation:
 - Apply pricing nightly (automatic calculation and sync to all channels)
 - Manual override: host can override AI-recommended price
 - A/B testing: test different pricing strategies and see impact
- Transparency:
 - Show why a given price was set (e.g., "high demand weekend, +20%")
 - Show historical pricing trends
 - Forecast revenue impact of pricing change

API Endpoints:

- POST /revenue-settings – Configure pricing rules
- GET /revenue-forecast?date_range= – Revenue forecast based on current pricing
- PATCH /listings/{id}/price?date= – Override specific date price
- POST /revenue-settings/event-pricing – Add event-based modifier
- GET /revenue-analytics – Historical pricing and revenue correlations

Data Model:

```
PricingRule {
  id: UUID
  property_id: UUID
  base_rate: decimal
  min_price: decimal
  max_price: decimal
  rules: {
    rule_type: enum [LOS_DISCOUNT, EARLY_BIRD, LAST_MINUTE, WEEKEND_MODIFIER,
    EVENT_MODIFIER, OCCUPANCY_BASED, COMPETITOR_BASED]
    params: {
      los_days?: integer
      discount_percent?: float
      days_ahead_threshold?: integer
      event_date?: date
      boost_percent?: float
      occupancy_threshold?: float
      competitor_offset_percent?: float
    }
  }
}[]
ai_optimization_enabled: bool
last_updated: timestamp
}
```

```
PricingHistory {
  id: UUID
  property_id: UUID
  date: date
  calculated_rate: decimal
  override_rate?: decimal
  applied_rate: decimal
  demand_signal: string # "high", "medium", "low"
  occupancy_nearby_percent?: float
  reason: string # "event modifier +30%", "occupancy-based", etc.
}
```

revenue_actual?: decimal # Actual revenue booked at this rate
}

Automation & Triggers:

- Nightly pricing calculation (11 PM in property timezone)
- Sync updated prices to all channels within 5 min
- Alert manager if price change >20% from baseline
- Auto-capitalize on event spikes (detect events via calendar API)

Permissions & Roles:

- Host: enable/disable optimization, set rules
- Revenue Manager: configure all pricing rules
- Manager: override prices, view analytics

Integrations:

- External pricing data providers (AirDNA, Airdna, ToursByLocals for event detection)
- Google Calendar API (detect major events, holidays)
- Weather API (for demand signal on beach properties, ski resorts)
- Airbnb API (for comparable listing data)

5.3.2 Payment Solutions & Guesty Pay™

Overview:

Payment orchestration platform for collecting booking payments, security deposits, and upsells with PCI compliance, fraud detection, and automated reconciliation.

User Stories:

- As a host, I want to collect payment from guests automatically so I don't chase payments
- As a guest, I want to pay securely with my card so I feel safe
- As a host, I want to see payment status clearly so I know if booking is confirmed
- As a property manager, I want to collect security deposits so I have collateral against damage

Functional Requirements:

- Payment methods:
 - Credit/debit cards (Visa, Mastercard, Amex)
 - Bank transfers (ACH, SEPA)
 - PayPal, Apple Pay, Google Pay (via payment processor)
 - Split payments: collect partial upfront, balance before check-in
- Charge types:
 - Security deposit (held until post-checkout, released if no damage)
 - Nightly rate (charged upfront or on arrival)
 - Fees (Guesty fees, service fees, taxes)
 - Upsells (add-ons like cleaning, insurance, late checkout)
 - Refunds (full or partial)
- Security:
 - PCI DSS Level 1 compliance

- Tokenization: store payment tokens instead of full cards
 - 3D Secure (SCA/3DS) for stronger authentication
 - Fraud detection: flag suspicious transactions
 - Encryption: all sensitive data encrypted in transit and at rest
- Reconciliation:
 - Automatic settlement matching with bank account
 - Handle refunds and chargebacks
 - Detailed payment ledger
- Payment status tracking:
 - Pending, processing, succeeded, failed, refunded, disputed
 - Automatic retry on failure (3 attempts)
 - Manual retry option for manager
- Currency handling:
 - Multi-currency support (convert rates daily)
 - Display prices in guest's local currency (optional)

API Endpoints:

- POST /payments/checkout – Initiate payment checkout session
- POST /payments/{id}/capture – Capture held charge
- POST /payments/{id}/refund – Process refund
- GET /payments/{id}/status – Check payment status
- POST /payments/webhook – Handle provider webhooks (bank settlement, disputes)

Data Model:

```

Payment {
  id: UUID
  reservation_id: UUID
  guest_id: UUID
  amount: decimal
  currency: string
  payment_type: enum [BOOKING_RATE, SECURITY_DEPOSIT, FEE, UPSELL]
  method: enum [CARD, BANK_TRANSFER, PAYPAL, APPLE_PAY, GOOGLE_PAY]
  status: enum [PENDING, PROCESSING, AUTHORIZED, CAPTURED, FAILED, REFUNDED,
DISPUTED]
  payment_method_token: encrypted_string # Tokenized card or bank account
  charge_reference: string # Stripe charge ID, etc.
  captured_at?: timestamp
  failed_reason?: string
  attempts: integer
  last_attempt_at?: timestamp
}

Refund {
  id: UUID
  payment_id: UUID
  amount: decimal
  reason: enum [DAMAGE, CANCELLATION, OVERPAYMENT, DISPUTE_RESOLUTION, OTHER]
  status: enum [PENDING, PROCESSED, FAILED]
  processed_at?: timestamp
}

```

```
PaymentReconciliation {  
  id: UUID  
  bank_transaction_id: string  
  amount: decimal  
  date: date  
  matched_payments: Payment[]  
  discrepancies?: string # If reconciliation doesn't match  
}
```

Automation & Triggers:

- Charge upfront at booking confirmation (configurable)
- Send payment reminder if payment pending 48 hours before check-in
- Auto-retry failed payments daily for 3 days
- Flag chargebacks for dispute review
- Auto-reconcile successfully processed payments against bank deposits

Permissions & Roles:

- Host: view payment status
- Manager: manage payment settings, process refunds
- Finance: view all payments and reconciliation
- Support: process refunds with approval

Integrations:

- Stripe (primary payment processor)
- PayPal (alternative processor)
- Square (alternative)
- Wise (multi-currency transfers)
- Bank integration: ACH, SEPA, wire transfers
- Compliance: tokenization via PCI DSS Level 1 solution

5.3.3 Trust Accounting

Overview:

Hospitality-specific accounting module managing owner balances, escrow, revenue reconciliation, payouts, and tax reporting aligned with STR regulations.

User Stories:

- As an owner, I want to see exactly how much I'm earning after all fees so I trust my property manager
- As a property manager, I want automated accounting so I don't manually reconcile payments
- As a finance officer, I want tax-compliant reporting so I meet filing requirements
- As an owner, I want clear statements showing revenue, expenses, and net payout

Functional Requirements:

- Owner ledger:
 - Track each owner's account balance (liability/owed amount)
 - Revenue in: booking proceeds, deposits returned (no damage)

- Deductions: Guesty fees, service fees, taxes, damage payouts, owner distributions
 - Running balance: always shows owner what's owed
- Escrow management:
 - Security deposits held in escrow (separate from operating funds)
 - Release deposits post-checkout if no damage
 - Hold dispute resolution period if damage claimed
 - Automatic release after expiration
- Tax handling:
 - Collect guest taxes (local occupancy taxes, VAT, etc.)
 - Track by jurisdiction
 - Generate tax reports (amount collected, remittance dates)
 - Support multiple tax rates per property
- Payout management:
 - Automatic payout schedules: weekly, bi-weekly, monthly (configurable per owner)
 - Manual payout: process on-demand
 - Minimum payout threshold (e.g., don't payout if <\$50)
 - Bank transfer, PayPal, Wise as payout methods
 - Track payout status: pending, in-transit, completed
- Reconciliation:
 - Daily: match bookings to payments collected
 - Weekly: identify discrepancies
 - Handle refunds, chargebacks, failed payments
 - Generate reconciliation reports
- Compliance:
 - Track tax identification numbers per owner
 - GDPR-compliant data retention
 - Audit trail: all account changes logged
 - 1099 reporting (US) or equivalent tax forms

API Endpoints:

- GET /accounting/owner/{id}/statement?date_range= – Owner statement
- POST /accounting/payout – Initiate payout
- GET /accounting/tax-report?jurisdiction= – Tax compliance report
- GET /accounting/escrow/{id}/status – Check escrow balance
- POST /accounting/reconciliation – Run reconciliation

Data Model:

```
OwnerLedger {
  id: UUID
  owner_id: UUID
  property_id: UUID
  period_start: date
  period_end: date
  revenue_gross: decimal
  revenue_net: decimal
  deductions: {
    guesty_fees: decimal
    service_fees: decimal
  }
}
```



```
taxes_collected: decimal
damage_payouts: decimal
chargebacks: decimal
}
balance_owed: decimal
last_payout_date?: date
last_payout_amount?: decimal
}
```

```
Escrow {
id: UUID
reservation_id: UUID
property_id: UUID
owner_id: UUID
amount: decimal
currency: string
held_from: date
release_after: date # Post-checkout + dispute period
status: enum [HELD, RELEASED, DISPUTED, RELEASED_WITH_DEDUCTION]
damage_claim_id?: UUID
released_at?: timestamp
}
```

```
TaxTrack {
id: UUID
property_id: UUID
jurisdiction: string
period: date
guest_tax_collected: decimal
tax_rate: float
remittance_due_date: date
remittance_status: enum [PENDING, SUBMITTED, COMPLETED]
remittance_reference?: string
}
```

```
Payout {
id: UUID
owner_id: UUID
properties: UUID[]
amount: decimal
currency: string
method: enum [BANK_TRANSFER, PAYPAL, WISE]
status: enum [PENDING, PROCESSING, COMPLETED, FAILED]
requested_at: timestamp
processed_at?: timestamp
reference_id?: string # Bank transaction ID, etc.
}
```

```
AccountingEntry {
id: UUID
ledger_id: UUID
entry_date: date
}
```

```
type: enum [BOOKING_REVENUE, REFUND, FEE, TAX, PAYOUT, DAMAGE_DEDUCTION, CHARGEBACK]
amount: decimal
description: string
reference_id?: UUID # reservation ID, payment ID, etc.
created_at: timestamp
}
```

Automation & Triggers:

- Daily: calculate owner balances after new bookings/payments
- Weekly: generate statements for owners
- Nightly: reconciliation run
- Automatic payout on schedule (e.g., every Friday if >\$100 balance)
- Hold escrow for 14 days post-checkout, then auto-release if no damage claim
- Generate 1099 forms (Jan 31) for US owners

Permissions & Roles:

- Owner: view own statements, request payouts
- Property Manager: manage accounts, approve payouts
- Finance: run reports, reconciliation, handle disputes
- Accountant: view for external audit (with permission)
- Support: troubleshoot payment issues

Integrations:

- Stripe Connect (payouts, escrow)
- Wise API (multi-currency payouts)
- Bank APIs (ACH settlement, verification)
- Tax compliance providers (TurboTax API, tax authorities)
- QuickBooks/Xero (export accounting entries)

5.3.4 Owners Portal

Overview:

Dedicated portal for property owners to view performance, financials, and reservations, reducing support tickets and building transparency.

User Stories:

- As an owner, I want to see my property's bookings and revenue so I verify management quality
- As an owner, I want to block owner stays so I can block dates when I visit
- As an owner, I want clear financial statements so I know exactly what I'm earning
- As an owner, I want to request payouts so I get paid on my schedule

Functional Requirements:

- Dashboard:
 - YTD revenue, occupancy %, upcoming bookings (30 days)
 - Monthly revenue trend chart
 - Top performing dates/seasons

- Reservations: view upcoming bookings, guest info, check-in/out dates
- Calendar: view availability, owner stays, blocked dates; ability to add owner stay dates
- Financial statements:
 - Monthly breakdown: revenue, fees, payouts
 - YTD summary: cumulative performance
 - Detailed line items: what fees were charged, why
 - Historical comparison: same month year-over-year
- Payouts: view payout history, request manual payout
- Reports: download PDF statements, export data for accounting
- Communications: message inbox (view-only, escalate issues to manager)
- Settings: payment method, contact info, notification preferences

API Endpoints:

- GET /owners/{id}/dashboard – Dashboard data
- GET /owners/{id}/statement?month= – Monthly statement
- POST /owners/{id}/owner-stays – Block owner stay dates
- POST /owners/{id}/payout-request – Request payout

Data Model:

```
OwnerPortalAccess {
  owner_id: UUID
  property_id: UUID
  password_hash: encrypted_string
  email: string
  phone: string
  notification_preferences: {
    email_weekly_digest: bool
    email_payment_notifications: bool
    sms_booking_alerts: bool
  }
  created_at: timestamp
  last_login: timestamp
}
```

```
OwnerStay {
  id: UUID
  owner_id: UUID
  property_id: UUID
  check_in: date
  check_out: date
  purpose?: string
  blocked_from_booking: bool
}
```

Automation & Triggers:

- Send weekly digest email with YTD metrics
- Alert owner when payment is processed
- Send monthly statement on first of month
- Reminder if owner hasn't logged in 30 days

Permissions & Roles:

- Owner: view own property dashboards and statements
- Manager: can grant owner access to properties
- Owner-level permissions: no access to manager settings or other owners

Security:

- Login via email/password or SSO (Google, Microsoft)
 - 2FA optional
 - Session timeout: 30 minutes of inactivity
-

5.3.5 Open API

Overview:

RESTful API enabling third-party developers, integrations, and custom tools to access and manipulate Guesty data (reservations, listings, messages, tasks, etc.).

User Stories:

- As a developer, I want to access Guesty data via API so I can build custom integrations
- As an integration partner, I want webhooks so I can react to booking events in real-time
- As a developer, I want OAuth 2.0 authentication so I can build multi-tenant apps
- As a data analyst, I want to export raw data for BI so I can build custom reports

Functional Requirements:

- Authentication:
 - OAuth 2.0 for third-party apps (authorization code flow)
 - API keys for personal use / integrations
 - Scopes: read, write per resource (reservations, listings, payments, etc.)
- Endpoints (core resources):
 - Reservations: CRUD, filtering, bulk operations
 - Listings: CRUD, rates, availability
 - Guests: profiles, communication history
 - Messages: read, send, search
 - Tasks: CRUD, status updates
 - Payments: view, refund
 - Owner statements: read
 - Accounting: ledger, escrow
- Webhooks:
 - Events: reservation.created, reservation.confirmed, check_in, check_out, payment.received, message.received, review.posted
 - Retry mechanism: retry failed deliveries with exponential backoff
 - Signature verification: HMAC-SHA256 signature for security
- Rate limiting: 1000 requests/hour for standard API key
- Pagination: cursor-based pagination for large result sets
- Filtering: query parameters for advanced filtering (date ranges, status, channel, etc.)
- Batch operations: bulk import/update via endpoints
- Documentation: OpenAPI 3.0 spec, auto-generated docs with code samples

API Design (Examples):

GET /api/v1/reservations

GET /api/v1/reservations/{id}

POST /api/v1/reservations

PATCH /api/v1/reservations/{id}

DELETE /api/v1/reservations/{id}

GET /api/v1/listings/{id}/rates?date_from=&date_to=

PATCH /api/v1/listings/{id}/rates

GET /api/v1/messages?reservation_id=

POST /api/v1/messages

POST /api/v1/webhooks # Register webhook

GET /api/v1/webhooks

DELETE /api/v1/webhooks/{id}

Data Model (API):

ApiKey {

id: UUID

user_id: UUID

name: string

key_hash: encrypted_string

scopes: string[] # [reservations.read, reservations.write, payments.read, ...]

rate_limit_rpm: integer

created_at: timestamp

last_used: timestamp

active: bool

}

Webhook {

id: UUID

user_id: UUID

url: string

events: string[] # [reservation.created, payment.received, ...]

secret: encrypted_string # For HMAC signature

active: bool

retry_policy: {max_attempts: integer, backoff_multiplier: float}

}

WebhookEvent {

id: UUID

webhook_id: UUID

event_type: string

resource_id: UUID

payload: JSON

delivered_at?: timestamp

attempts: integer

last_error?: string

}

Permissions & Roles:

- User: create and manage own API keys and webhooks
- Third-party app: OAuth scopes limit access
- Support: can revoke keys if needed

Security:

- API keys: rotate regularly, disable old ones
- Webhook signatures: HMAC-SHA256 to verify authenticity
- Rate limiting: prevent abuse
- IP whitelisting: optional
- Audit logging: all API calls logged with user/app identification

5.3.6 Guesty Capital™ (Optional Add-on)

Overview:

Financing product offering working capital or growth capital to hosts based on revenue visibility and booking pipeline in Guesty platform.

User Stories:

- As a growing host, I want to expand my portfolio but need capital so I apply for a loan based on my Guesty data
- As a host, I want to fund property upgrades without withdrawing profits so I take a short-term loan
- As a manager, I want to finance growth and see ROI on the platform

Functional Requirements:

- Eligibility assessment:
 - Minimum 6 months of Guesty history
 - Minimum revenue threshold (\$X/month)
 - Good occupancy rate (>60%)
 - Positive booking trend (month-over-month growth)
- Loan products:
 - Short-term (working capital): 3–12 months, interest-based
 - Medium-term (growth): 12–36 months, interest-based
 - Revenue-share: lender takes % of future revenue instead of fixed repayment
- Underwriting:
 - Automated based on Guesty data (revenue, occupancy, payment history)
 - Manual review for edge cases
 - Credit check (via third-party provider)
- Application process:
 - Pre-approval: instant decision based on Guesty metrics
 - Full application: submit via portal, provide business plan
 - Approval: review, decision, funding
- Funding: wire transfer to host or owner
- Repayment:
 - Fixed monthly installments (traditional loan)
 - Revenue share: automatic withholding from Guesty payouts
- Dashboard: track loan balance, repayment schedule, interest accrued

API Endpoints:

- GET /capital/pre-approval – Check eligibility
- POST /capital/application – Submit application
- GET /capital/loans/{id} – View loan details
- GET /capital/repayment-schedule – View upcoming payments

Data Model:

```
CapitalApplication {
  id: UUID
  user_id: UUID
  status: enum [DRAFT, SUBMITTED, UNDER_REVIEW, APPROVED, FUNDED, REJECTED]
  loan_amount_requested: decimal
  loan_term_months: integer
  purpose: string
  submitted_at?: timestamp
  approved_at?: timestamp
  funded_at?: timestamp
}
```

```
Loan {
  id: UUID
  application_id: UUID
  user_id: UUID
  amount: decimal
  interest_rate: float
  term_months: integer
  monthly_payment: decimal
  remaining_balance: decimal
  status: enum [ACTIVE, PAID_OFF, DEFAULT]
  funding_date: date
  maturity_date: date
  next_payment_date: date
}
```

```
LoanRepayment {
  id: UUID
  loan_id: UUID
  amount: decimal
  date: date
  method: enum [BANK_TRANSFER, GUESTY_WITHHOLD]
  status: enum [PENDING, PROCESSED, FAILED]
}
```

Automation & Triggers:

- Nightly: calculate next payment due, withhold from Guesty payouts if revenue-share loan
- Alert if payment >5 days late

Integrations:

- Credit bureaus (Equifax, Experian, TransUnion)
- Bank for fund transfer

- Underwriting platform (automated decision engine)
-

5.3.7 Travel Protection (Add-on)

Overview:

Trip insurance product offered to guests at booking, covering trip cancellations, delays, and emergencies.

User Stories:

- As a host, I want to offer guests trip insurance as an upsell so I increase per-booking revenue
- As a guest, I want to buy trip insurance so I'm protected if my plans change
- As a host, I want to see which guests bought insurance so I know claim risk

Functional Requirements:

- Offer at booking: include trip insurance as optional add-on in booking flow
- Coverage:
 - Trip cancellation (reimburse booking if cancels due to illness, emergency, etc.)
 - Trip delay (cover costs if delayed >X hours)
 - Emergency assistance (medical, evacuation)
 - Baggage loss
- Pricing: flat fee or percentage of booking (configurable)
- Claims process:
 - Guest files claim with supporting docs (medical cert, cancellation reason, receipts)
 - Guesty or partner reviews claim
 - Approval and reimbursement
- Integration with Guesty booking flow: show coverage details, accept/decline in guest app

API Endpoints:

- POST /add-ons/travel-protection/{id} – Add to booking
 - POST /claims/travel-protection – File claim
 - GET /claims/travel-protection/{id} – Check claim status
-

6. User Journeys & Workflows

6.1 Host Onboarding & First Booking

Flow:

1. Host signs up via web/mobile
2. Select plan (Lite/Pro/Enterprise)
3. Connect first property: basic info (address, bedrooms, photos)
4. Connect first OTA: OAuth to Airbnb/Booking.com
5. Sync existing listings or create new
6. Configure guest app branding
7. Set up automation rules (optional)
8. Receive first booking → system confirms, sends pre-arrival sequence to guest

9. Guest checks in → host monitors via app, receives alerts
10. Post-checkout → task auto-created, payment reconciled, guest asked for review

6.2 Revenue Optimization Workflow (Manager)

Flow:

1. Log in to Guesty dashboard
2. Review occupancy/ADR trends via analytics
3. Identify underperforming periods (e.g., low occupancy mid-week)
4. Review competitor pricing in same area
5. Adjust base rates and seasonal modifiers
6. Enable PriceOptimizer™ with conservative parameters
7. Monitor rates applied over next 30 days
8. Track revenue impact (ADR, RevPAR)
9. Fine-tune rules based on results
10. Apply winning rules to other properties

6.3 Cleaning & Maintenance Workflow

Flow:

1. Guest checks out at 11 AM
2. Task automatically created: "Post-checkout cleaning, due 2 PM"
3. Notification sent to assigned cleaner (SMS/app)
4. Cleaner receives task, views checklist and photos
5. Cleaner performs cleaning, uploads completion photos
6. Task marked complete, quality review triggered (optional)
7. Manager reviews photos/status
8. Property marked available for next guest
9. Pre-arrival inspection task auto-created if enabled

6.4 Payment Collection & Reconciliation Workflow

Flow:

1. Booking confirmed → payment due 14 days before check-in
 2. Guest receives payment link (email/SMS/guest app)
 3. Guest pays via Stripe → payment captured
 4. Confirmation sent to guest and host
 5. Daily reconciliation: match deposits to Guesty payments
 6. Weekly: generate owner statement
 7. End of month: calculate payouts
 8. Automatic payout: transfer to owner's bank or PayPal
 9. Owner receives payout notification and detailed statement
-

7. Non-Functional Requirements

7.1 Performance & Scalability

- **Latency:** API response time <500ms (p95)
- **Throughput:** Support 10,000+ concurrent users, 1M+ reservations
- **Availability:** 99.9% uptime SLA
- **Scalability:** Horizontally scalable backend (microservices, load balancing)
- **Database:** Optimized queries, indexing, read replicas for analytics

7.2 Security & Compliance

- **Authentication:** OAuth 2.0, JWT tokens, 2FA support
- **Encryption:** TLS 1.3 in transit, AES-256 at rest
- **PCI DSS:** Level 1 compliance for payment processing
- **GDPR:** Consent management, data export, right to be forgotten
- **Data retention:** Configurable per jurisdiction
- **Audit logging:** All user actions logged, immutable audit trail
- **Permissions:** Role-based access control (RBAC) with granular scopes

7.3 Reliability & Disaster Recovery

- **Backup:** Daily backups, multi-region replication
- **RTO:** 4 hours (Recovery Time Objective)
- **RPO:** 1 hour (Recovery Point Objective)
- **Monitoring:** Real-time alerts on errors, latency, failed syncs
- **Logging:** Centralized logging, searchable across services

7.4 Usability & Accessibility

- **Responsive design:** Mobile-first, works on all screen sizes
- **Accessibility:** WCAG 2.1 AA compliance, keyboard navigation, screen reader support
- **Localization:** Support multiple languages (at least 5: EN, ES, FR, DE, IT)
- **Onboarding:** Interactive walkthrough for new users
- **Help & documentation:** In-app help center, video tutorials, email support

7.5 Integration & Interoperability

- **API-first:** Core features exposed via REST API
- **Webhooks:** Real-time event delivery with retries
- **Standards:** OpenAPI 3.0, JSON, OAuth 2.0
- **Rate limits:** Documented and enforced
- **Versioning:** Backward-compatible API versions

8. Success Metrics & KPIs

8.1 Product Metrics

Metric	Target	Measurement
User Adoption	50% of new signups complete onboarding in 48h	Days to first booking / property setup
Feature Usage	80% of Pro users use revenue management	Monthly active feature use
Channel Sync	99.5% of rates sync successfully	Failed sync count / total syncs
Message Response Time	<1h average (vs. 4h+ before)	Timestamp from message to response
Occupancy Impact	+15% avg occupancy (via automation, direct bookings)	Booked nights / available nights
Revenue Impact	+20% ADR (via dynamic pricing)	Average daily rate YoY

8.2 Business Metrics

Metric	Target
Customer Retention	95% annual retention for Pro plan
NPS (Net Promoter Score)	>50 NPS
Support Ticket Resolution	95% resolved in <24h
Churn Rate	<5% monthly

8.3 Financial Metrics

Metric	Target
ARPU (Annual Recurring Revenue Per User)	Pro: \$2,500; Enterprise: \$50,000+
CAC (Customer Acquisition Cost)	<ARPU/3 (payback within 3 months)
LTV (Lifetime Value)	LTV:CAC ratio >3:1

9. Implementation Roadmap (Phases)

Phase 1: MVP (Months 1–3)

- Core features: Multi-calendar, unified inbox, channel manager, basic automation
- Payment basics: Stripe integration, direct reservations
- Target: Serve Lite segment (1–3 listings)

Phase 2: Pro Features (Months 4–6)

- Revenue management: Dynamic pricing rules, competitor benchmarking
- Task management: Auto-generation, mobile app
- Guest communications: AI suite, guest app
- Owner portal: Financial statements, payouts
- Target: Serve Pro segment (4–199 listings)

Phase 3: Enterprise Scale (Months 7–9)

- Enterprise Hub: Multi-brand, permissions, consolidated reporting
- Advanced integrations: Marketplace, API, webhooks
- Guesty Capital: Financing product
- Shield Suite: Comprehensive risk mitigation
- Target: Serve Enterprise segment (200+)

Phase 4: Network Effects & Marketplace (Months 10–12)

- Integration marketplace: list third-party tools
- Community features: host knowledge sharing
- Advanced analytics & benchmarking
- Expansion to adjacent markets (institutional investors, PMCs)

10. Competitive Advantages

1. **Unified Operations:** Single control plane vs. fragmented tools (Airbnb Manager + Booking Partner Manager + separate task app + separate accounting)
 2. **AI-Driven:** AI message suggestions, dynamic pricing, risk scoring reduce manual work
 3. **Direct Booking:** Native website builder + payment processing keeps commission in ecosystem
 4. **Trust & Compliance:** Native accounting (trust ledger, escrow, tax) vs. manual reconciliation
 5. **Ecosystem:** Broad marketplace + Open API allows endless customization
 6. **Scalability:** Same platform supports solopreneurs to enterprises (1 to 10,000+ listings)
-

11. Risk Mitigation

Risk	Mitigation
Channel API changes (Airbnb, Booking changes rates/policies)	Maintain active partnerships, rapid dev response, fallback to manual entry
Payment processor failures	Multi-processor support (Stripe + PayPal), retry logic, escrow for unprocessed amounts
Data loss or breach	Multi-region backup, encryption, PCI DSS compliance, incident response plan
Regulatory changes (tax, STR restrictions)	Legal review, compliance team, configurable rule engine for jurisdiction-specific rules
Churn from feature bloat	Regular UX testing, feature flags, gradual rollout, focused on core pain points
Competitor copy or acquisition	Strong network effects (ecosystem), brand loyalty, continuous innovation

12. Future Expansion Opportunities

1. **Institutional Investor Dashboard:** Specialized for portfolio investors managing 1,000+ units
 2. **Co-hosting Platform:** Enable hosts to hire experienced co-hosts via Guesty Marketplace
 3. **Dynamic Sourcing:** Supply-side tools for property investors to identify and acquire high-yield properties
 4. **STR Secondary Market:** Trading platform for STR inventory rights (experimental)
 5. **Financing Expansion:** Offer mortgage/acquisition financing based on revenue predictability
 6. **Expansion to Traditional Hotels:** Adapt PMS for independent hotels and small chains
 7. **Corporate Housing:** Serve corporate short-term housing providers with compliance and billing features
-

Appendix: Glossary

- **ADR (Average Daily Rate):** Total revenue / number of booked nights
- **RevPAR (Revenue Per Available Room):** Total revenue / total available room-nights
- **LOS (Length of Stay):** Number of nights per reservation
- **OTA (Online Travel Agency):** Platform like Airbnb, [Booking.com](#), Vrbo
- **PMS (Property Management System):** Software for managing rental properties
- **Occupancy Rate:** Booked nights / total available nights
- **Direct Booking:** Reservation made outside OTA (website, phone, email)
- **Trust Accounting:** Accounting model where host/owner funds are held in escrow and reconciled transparently
- **Turnover:** Time between checkout and next check-in (cleaning period)
- **Breach:** Gap in calendar with no reservation
- **GDPR:** General Data Protection Regulation (EU privacy law)
- **SLA:** Service Level Agreement (performance guarantee)
- **API:** Application Programming Interface (technical interface for data access)
- **Webhook:** Automatic push notification when event occurs
- **OAuth:** Standard authentication protocol for third-party app access
- **Tokenization:** Converting sensitive data (card) into non-sensitive token

Document prepared for: Reverse-engineered competitive analysis and product specification

Use case: Building a first-class STR property management platform

Last updated: December 30, 2025